

# Optimizer Step Processing Dynamics in Apple MLX: A Comparative Analysis of AdamW, Muon, and GEMM-Based Optimizers

The pretraining and fine-tuning of large language models (LLMs) fundamentally revolve around the intricate trade-off between optimization sample efficiency and the computational overhead incurred per step. Historically, deep learning paradigms relied almost exclusively on first-order, element-wise optimizers, with Adam and its weight-decay variant, AdamW, serving as the undisputed industry standards. However, the rapidly expanding parameter counts of modern architectures have catalyzed the advent of matrix-aware, higher-order optimizers. Frameworks such as Muon (MomentUm Orthogonalized by Newton-Schulz) and GEMM-based preconditioned optimizers like SOAP (ShampooO with Adam in the Preconditioner's eigenbasis) have demonstrably fractured the AdamW consensus. These modern optimizers achieve target validation losses in significantly fewer training steps by exploiting the spectral geometry and Kronecker-factored structural correlations of neural network weight matrices<sup>1</sup>.

While these geometric advantages drastically reduce the total token count required for convergence, they simultaneously impose a severe computational penalty, effectively transforming the optimizer step from a memory-bandwidth-bound operation into a heavily compute-bound operation. This paradigm shift creates unique challenges and opportunities within the Apple Silicon ecosystem, specifically when utilizing the MLX framework. Apple's Unified Memory Architecture (UMA) physically co-locates the CPU and GPU memory spaces, drastically altering the latency, bandwidth, and execution dynamics of the forward, backward, and optimizer update passes<sup>4</sup>.

In specialized repositories and configurations dedicated to local, low-memory LLM training—such as the conceptual and practical implementations surrounding <https://github.com/dtelcore/Apple-MLX-llm-gpu-8/>—the choice of optimizer dictates the delicate balance between achievable batch size, peak memory consumption, and wall-clock training time<sup>6</sup>. Understanding the step processing times of AdamW (and its custom GPU-accelerated variants like AdamWGPU), Muon, and GEMM-based optimizers in MLX requires a rigorous architectural analysis of lazy evaluation computation graphs, Metal kernel dispatches, 8-bit quantization protocols, and advanced algebraic reductions such as the Gram Newton-Schulz algorithm.

## The Apple MLX Computational Architecture

To properly contextualize the performance profiles of various optimizers, it is imperative to deconstruct the execution mechanics of the Apple MLX framework. Unlike traditional deep learning frameworks such as PyTorch, which typically dispatch CUDA kernels eagerly to a discrete GPU, MLX is built fundamentally upon lazy (deferred) evaluation<sup>4</sup>. When a typical

optimizer update loop runs within MLX, the mathematical operations construct a directed acyclic computation graph in system memory but do not execute immediately on the hardware.

## **Lazy Evaluation and Graph Compilation**

Computation is only materialized when an explicit evaluation trigger is invoked, such as calling `mx.eval()`, printing an array to the console, or converting an MLX array to a Python scalar or NumPy array<sup>4</sup>. This deferred execution model is the cornerstone of step time optimization on Apple Silicon. It empowers the MLX compiler to perform extensive graph-level optimizations before any silicon is engaged. The compiler can seamlessly fuse element-wise operations, schedule work optimally across the CPU performance cores and the GPU, and eliminate redundant computations by tracking array dependencies<sup>4</sup>.

For a standard LLM training loop, the optimizer step calculates the updated parameters based on the gradients gathered during the backward pass. However, the computation graph remains entirely symbolic until a command like `mx.eval(model.parameters(), optimizer.state)` is explicitly executed<sup>8</sup>. By wrapping these symbolic graphs in the `mx.compile()` transform, MLX can Just-In-Time (JIT) compile the optimizer step into highly optimized Metal shaders<sup>4</sup>. The compilation process drastically reduces the Python interpreter overhead, caching the graph structure by input shape to ensure that subsequent optimizer steps execute with minimal CPU latency<sup>9</sup>. This is particularly critical for higher-order optimizers that execute hundreds of intermediate matrix operations per step.

## **Unified Memory Dynamics and Mixed Execution**

The unified memory architecture of Apple Silicon fundamentally alters the memory traffic dynamics of the optimizer step, differentiating it sharply from discrete NVIDIA or AMD GPU configurations. In traditional setups, explicit host-to-device transfers (e.g., `tensor.to('cuda')`) create severe PCIe bandwidth bottlenecks<sup>4</sup>. In contrast, MLX arrays reside in a shared physical memory pool accessible directly by both the CPU and the Metal-driven GPU<sup>4</sup>. Consequently, there is no serialization or device transfer cost incurred when shifting computations.

This architecture allows MLX to perform mixed-device execution asynchronously and with zero-copy overhead. Because there is no transfer cost, one can mix devices within a single computation block to maximize hardware utilization<sup>5</sup>. The framework releases the Python Global Interpreter Lock (GIL) during `mx.eval()` or `mx.async_eval()`, permitting background CPU threads to continue processing dataloaders or handling web requests while the GPU executes the heavy optimizer computations in the background<sup>5</sup>. For memory-bound optimizers like standard AdamW, the unified memory bandwidth sets the absolute ceiling on step time. For compute-bound optimizers like Muon, the parallel execution efficiency of the Metal General Matrix Multiplication (GEMM) kernels dictates the throughput.

## **Custom Metal Kernels and Math Modes**

To further accelerate custom optimizer logic, MLX allows researchers to bypass standard high-level array operations by writing custom Metal kernels via the `mlx.core.fast.metal_kernel` API<sup>10</sup>. These JIT-compiled kernels allow for precise threadgroup dispatching and shared memory utilization. When deploying optimizers in low-memory, 8-bit configurations typical of projects like Apple-MLX-llm-gpu-8, these custom kernels are utilized to fuse quantization, step

updates, and dequantization into a single execution pass<sup>6</sup>. Furthermore, MLX provides compiler flags such as `compile_options={"math_mode": "fast"}` or `"relaxed"`, allowing the Metal compiler to ignore strict IEEE compliance for special values (like NaN or -inf), thereby yielding higher ALU throughput during the dense floating-point operations characteristic of optimizer steps<sup>11</sup>.

## First-Order Optimization: AdamW and AdamWGPU

AdamW serves as the baseline against which all modern LLM optimizers are measured. As a first-order, element-wise adaptive optimizer, it calculates parameter updates by maintaining two exponentially decaying moments for each weight: the first moment (momentum) and the second moment (variance)<sup>12</sup>.

### The Memory-Bound Nature of AdamW

The update rule for AdamW is highly localized, applying scalar operations across flattened tensors. In the MLX environment, the step processing time for AdamW is almost entirely dictated by memory bandwidth rather than compute capability. For a parameter matrix of size  $m \times n$ , the AdamW step operates with an  $O(mn)$  time complexity<sup>14</sup>. Because it performs very few floating-point operations (FLOPs) per byte of data loaded from RAM, the arithmetic intensity is exceptionally low. The GPU's arithmetic logic units (ALUs) spend the vast majority of the step time idle, waiting for the massive first and second moment buffers to be fetched from unified memory and written back after the update.

This memory-bound nature is severely exacerbated when scaling to multi-billion-parameter models. In standard 32-bit floating-point precision, AdamW requires 8 bytes of optimizer state per parameter (4 bytes for the momentum buffer, 4 bytes for the variance buffer)<sup>13</sup>. For a 7-billion parameter model, the optimizer state alone consumes roughly 56 GB of memory, which easily exceeds the unified memory capacity of standard Apple Silicon machines unless advanced offloading or quantization techniques are employed<sup>13</sup>.

### AdamWGPU and 8-bit Quantized States

Configurations targeting local GPU constraints, such as the 8-bit regimes explored in frameworks resembling Apple-MLX-llm-gpu-8, actively seek to compress this optimizer state. Recent state-compression innovations, such as Gefen-X and FlashOptim, have demonstrated that the AdamW state can be quantized down to approximately 1 byte per parameter (or 7 bytes total including gradients)<sup>7</sup>. By mapping the variance and momentum tensors into heavily compressed lookup structures using companding functions, these implementations cut the optimizer memory footprint by up to a factor of eight while maintaining standard AdamW-level convergence trajectories<sup>7</sup>.

In MLX, an "AdamWGPU" implementation utilizes highly optimized custom Metal kernels to execute these compressed updates efficiently. Using `mlx.core.fast.metal_kernel`, the optimizer can perform state decompression, the element-wise AdamW scaling calculation, and re-compression entirely within the GPU's registers<sup>7</sup>. This prevents the intermediate 32-bit upcasting that characterizes naive 8-bit optimizer implementations, which would otherwise cause massive transient spikes in peak memory consumption<sup>7</sup>. Despite these profound memory optimizations, the wall-clock step time of AdamWGPU remains linear with respect to the

parameter count and continues to be bounded by the raw memory bandwidth of the Apple M-series architecture.

| Optimizer Profile       | Targeted Parameters | Algorithmic Complexity | Optimizer State Memory (per param) | Main Execution Bottleneck |
|-------------------------|---------------------|------------------------|------------------------------------|---------------------------|
| <b>AdamW (fp32)</b>     | All (1D & 2D)       | $O(mn)$                | 8 bytes (2 buffers)                | Unified Memory Bandwidth  |
| <b>Gefen-X (8-bit)</b>  | All (1D & 2D)       | $O(mn)$                | ~1 byte                            | Unified Memory Bandwidth  |
| <b>AdamWGPU (Fused)</b> | All (1D & 2D)       | $O(mn)$                | 4 to 8 bytes                       | Unified Memory Bandwidth  |
| <b>Lion</b>             | All (1D & 2D)       | $O(mn)$                | 4 bytes (1 buffer)                 | Unified Memory Bandwidth  |

## Lion and Signum: Memory-Efficient First-Order Alternatives

Before transitioning to matrix-aware optimizers, it is vital to acknowledge other first-order optimizers that achieve high throughput on Apple Silicon. The Lion optimizer (EvoLved Sign Momentum) simplifies the update step by applying the sign operation to the momentum, completely dropping the variance tracking required by AdamW<sup>16</sup>. By maintaining only a single momentum buffer, Lion matches the 4-byte-per-parameter memory footprint of Muon, while retaining the  $O(mn)$  linear time complexity of AdamW<sup>13</sup>. On Apple MLX, Lion executes faster per step than AdamW because it halves the memory traffic, though it requires meticulous learning rate tuning to prevent divergence<sup>8</sup>. Signum (Sign Momentum) operates on a similar principle, relying purely on the sign of the gradients to determine the descent direction, prioritizing speed and memory efficiency over the nuanced scaling provided by second-moment estimation<sup>19</sup>.

## The Orthogonalized Paradigm: Muon and Spectral Norm Geometry

While AdamW scales easily from an engineering perspective, its element-wise nature inherently

ignores the correlational structure between gradients and the geometric operator nature of neural network weight matrices<sup>1</sup>. LLM weights are not merely collections of independent scalars; they are linear transformations mapping latent spaces. This theoretical limitation has driven the rapid adoption of Muon (MomentUm Orthogonalized by Newton-Schulz), which has proven instrumental in training frontier sparse and dense models, including Kimi K2, GLM-5, and DeepSeek-V4<sup>14</sup>.

Muon operates on a fundamentally different geometric premise: it is explicitly designed to optimize two-dimensional weight matrices by performing steepest descent under the spectral norm<sup>14</sup>. Instead of scaling individual scalar elements by their historical variance, Muon treats the entire gradient matrix as a unified geometric operator. At each training step, Muon calculates the gradient, computes an exponentially moving average to form the momentum, and then applies a polar decomposition to orthogonalize the momentum matrix before applying the update to the weights<sup>13</sup>.

## The Mathematics of Muon and SVD Approximation

Mathematically, for a momentum matrix  $M_k \in \mathbb{R}^{n \times m}$  at step  $k$ , the ideal orthogonalized update is the polar component derived from the Singular Value Decomposition (SVD). If the SVD of the momentum is  $M_k = U \Sigma V^\top$ , the polar decomposition is:

$$\text{polar}(M_k) = M_k (M_k^\top M_k)^{-1/2} = U V^\top$$

By extracting only the orthogonal matrices  $U$  and  $V^\top$ , Muon effectively replaces the singular values of its momentum matrix with exactly one, producing an update with a perfectly flat singular spectrum<sup>14</sup>. This perfectly spatial decorrelation allows the neural network to navigate the loss landscape without the poor conditioning that typically plagues deep linear transformations.

Because computing exact SVDs during every optimizer step is computationally disastrous for large neural networks, Muon approximates the polar decomposition using the Newton-Schulz iteration<sup>14</sup>. The algorithm takes the Frobenius-normalized momentum matrix and repeatedly applies an odd-degree matrix polynomial to push its singular values toward 1<sup>20</sup>. A standard

Newton-Schulz iteration updates a matrix  $X_t$  as follows:

$$X_{t+1} = \frac{1}{2} X_t (3I - X_t^\top X_t)$$

In modern implementations, higher-degree polynomials (e.g., degree-5) are utilized to achieve convergence in fewer iterations<sup>14</sup>.

## The Computational Overhead on MLX

The theoretical superiority of Muon translates into profound sample efficiency. Empirically, on language modeling tasks and NanoGPT speedrunning benchmarks, Muon reaches target

validation losses up to 25% to 35% faster in terms of training tokens compared to AdamW<sup>12</sup>.

However, this token efficiency comes at a severe cost to the step processing time.

The Newton-Schulz iterations in Muon require dense matrix multiplications, fundamentally shifting the optimizer step from a memory-bound operation to an ALU-bound (compute-bound)

operation. For a weight matrix of dimensions  $m \times n$ , performing the Newton-Schulz iteration

costs  $O(mn^2)$  time assuming  $n \leq m$ <sup>14</sup>. In a typical MLX setup requiring five iterations of degree-5 polynomials, the optimizer must execute multiple rectangular matrix multiplications per layer, per step<sup>20</sup>.

On Apple Silicon, these GEMMs are routed through the Metal backend via

`mlx.core.fast.metal_kernel` or standard `mx.matmul` dispatches<sup>10</sup>. While MLX handles GEMMs efficiently, the sheer volume of floating-point operations introduces a massive latency spike at the end of the backward pass.

Furthermore, Muon's memory characteristics present a dichotomy. On one hand, it requires only a single momentum buffer per parameter, halving the optimizer state memory compared to full-precision AdamW (4 bytes vs 8 bytes)<sup>13</sup>. On the other hand, the Newton-Schulz iteration requires transient activation memory to hold the intermediate

matrix states of the polynomial terms, such as  $X_t^\top X_t$  and  $(X_t^\top X_t)^2$ <sup>14</sup>. In unified memory systems, this transient spike can trigger severe memory pressure, forcing the system to evict caches or page data, which subsequently elongates the step processing time and nullifies the benefits of the smaller resting optimizer state.

## Hardware-Aware Algebraic Reductions: Gram Newton-Schulz and Dion3

To mitigate the cubic scaling overhead of standard Muon on hardware accelerators, algorithmic revisions such as the Gram Newton-Schulz (GNS) algorithm and the Dion3 framework have been developed. These revisions dramatically alter the step time dynamics, transforming Muon from an impractically slow theoretical curiosity into a production-ready optimizer.

### Overcoming Rectangular Inefficiencies

The standard Newton-Schulz algorithm performs poorly on modern transformer architectures because the weight matrices are often highly rectangular. For instance, in fine-grained Mixture-of-Experts (MoE) layers or expanded Feed-Forward Networks (FFNs), dimensions often

take the shape of  $m \times n$  where  $m \gg n$  (e.g., an aspect ratio of 8)<sup>23</sup>. The runtime of standard Newton-Schulz is heavily dependent on the larger dimension, requiring up to 10 rectangular matrix multiplications for a standard five-step approximation<sup>14</sup>.

The Gram Newton-Schulz approach exploits algebraic equivalences to bypass these large rectangular GEMMs entirely. GNS recognizes that the polar decomposition can be computed by isolating the Gram matrix of the momentum. Instead of iterating directly on the massive

rectangular matrix  $X \in \mathbb{R}^{n \times m}$ , GNS first computes the small, square, symmetric Gram

matrix  $R_0 = XX^\top \in \mathbb{R}^{n \times n}$ <sup>14</sup>. It then executes the Newton-Schulz-style polynomial iterations entirely within the much smaller  $n \times n$  subspace to find a matrix  $Q_t$  that converges to the inverse square root  $(XX^\top)^{-1/2}$ <sup>23</sup>. Only at the final step is this square matrix multiplied back against the rectangular matrix to yield the orthogonalized update.

## FLOP Reduction and Symmetric GEMMs in MLX

This restructuring yields profound performance advantages for MLX implementations:

1. **Algorithmic FLOP Reduction:** The theoretical floating-point operation (FLOP) count drops from approximately  $T(3\alpha + 1)n^3$  in the standard method to just  $(4T + 3\alpha - 3)n^3$  in GNS, where  $\alpha = m/n$  is the aspect ratio of the matrix and  $T$  is the number of iterations<sup>23</sup>. For transformer projection matrices with an aspect ratio of 8, GNS reduces the raw computational workload by up to 50%<sup>23</sup>.
2. **Symmetric GEMM Exploitation:** Because the Gram matrix  $XX^\top$  and its subsequent polynomial powers are perfectly symmetric by definition, specialized symmetric GEMM kernels can be deployed<sup>14</sup>. Rather than calculating the full dense output matrix, a symmetric kernel computes only the lower triangular portion and seamlessly reflects the results to the upper triangular portion<sup>14</sup>.

When executed on Apple Silicon via MLX, Gram Newton-Schulz transforms the Muon step from a prohibitive bottleneck into a highly optimized subroutine. Although writing custom symmetric GEMM kernels in Metal requires precise threadgroup management to avoid divergent branching, the net result is a Muon step that executes in roughly half the wall-clock time of a naive Muon implementation<sup>10</sup>.

## Stability and the Dion3 Megabatching Protocol

However, GNS is highly susceptible to numerical instability when utilizing low-precision arithmetic formats (like bfloat16), which are standard in MLX to maximize memory bandwidth<sup>14</sup>. To address this, implementations employ "restarts"—a stabilization technique where the Gram matrix is periodically reconstructed from the intermediate rectangular state to flush out accumulated floating-point errors and eliminate spurious negative eigenvalues<sup>14</sup>.

Further optimizations are found in frameworks like Dion3, which introduces megabatching and fractional row updates to Muon<sup>23</sup>. By defining a hyperparameter  $f \in (0, 1]$  representing the fraction of rows to update, Dion3 targets only a subset of the matrix rows featuring the largest  $\ell_1$  norms<sup>23</sup>. This reduces the dimensionality of the Gram matrix even further, heavily compressing the communication overhead and compute requirements, which is vital when distributing MLX workloads across multiple Apple Silicon nodes via Thunderbolt RDMA (as seen in specific MLX cluster benchmarks)<sup>23</sup>.

# Kronecker Preconditioning: SOAP and Shampoo

## Dynamics

Beyond spectral orthogonalization, GEMM-based optimizers utilizing Kronecker-factored preconditioning represent another frontier in LLM training efficiency. The Shampoo optimizer, and its modern derivative SOAP (ShampoO with Adam in the Preconditioner’s eigenbasis), aim to precondition the neural network gradients using second-order statistical information without explicitly forming the prohibitively massive inverse Hessian matrix<sup>3</sup>.

SOAP approximates the Fisher information matrix using the Kronecker product of two smaller, more manageable matrices. These matrices represent the statistics of the forward-pass activations and the backpropagated errors, respectively<sup>29</sup>. By projecting the gradients into this eigenspace before applying Adam-style exponential moving average updates, SOAP achieves a highly effective descent direction that navigates the loss landscape optimally, avoiding the sharp, unstable local minima to which first-order methods like AdamW often fall victim<sup>3</sup>.

## Amortized Eigendecomposition Step Times

The step processing time for SOAP introduces entirely different latency constraints compared to Muon. While Muon applies the Newton-Schulz approximation iteratively at every single training

step, SOAP requires the precise computation of inverse  $p$ -th roots of the preconditioner matrices (often executed via exact SVD or Schur decompositions). Because performing an exact eigendecomposition at every step is catastrophically slow, SOAP implementations amortize this computational cost by calculating the eigenbasis infrequently—typically updating the preconditioners only every 100 to 500 steps<sup>28</sup>.

In the MLX framework, this amortization creates a highly uneven step processing time profile. The vast majority of optimizer steps act similarly to a standard matrix projection, executing rapidly. However, the specific steps where the eigenbasis is refreshed encounter massive latency spikes.

## Large-Batch Scaling

Despite the uneven step times, SOAP scales exceptionally well at extremely large global batch sizes. In benchmarks testing Mixture-of-Experts (MoE) architectures up to 72 billion parameters, both Muon and SOAP maintained token-efficient convergence at global batch sizes reaching up to 100 million tokens, whereas AdamW’s performance degraded significantly past critical batch thresholds<sup>1</sup>. This makes SOAP highly attractive for massive-scale cluster pretraining. However, for local, fast-iterative fine-tuning loops on single Apple Silicon nodes (such as the setups targeted by Apple-MLX-llm-gpu-8), the unpredictable, high-latency eigendecomposition steps can stall the MLX evaluation pipeline, making Muon’s consistent step time slightly more preferable for responsive, interactive training.

| Optimizer | Algorithmic Mechanism | Convergence Efficiency | Compute Overhead Profile | Primary MLX Use Case |
|-----------|-----------------------|------------------------|--------------------------|----------------------|
|           |                       |                        |                          |                      |

|                  |                            |                         |                                   |                              |
|------------------|----------------------------|-------------------------|-----------------------------------|------------------------------|
| <b>Muon</b>      | Spectral Norm Descent      | High (+25% faster)      | Consistent per step ( $O(mn^2)$ ) | Dense matrix fine-tuning     |
| <b>Gram Muon</b> | Symmetric Newton-Schulz    | High (+25% faster)      | Consistent (up to 50% FLOP drop)  | Highly rectangular MoEs/LoRA |
| <b>SOAP</b>      | Kronecker Preconditioning  | Very High (+30% faster) | Spiky (Amortized SVD)             | Large batch pretraining      |
| <b>MONA</b>      | Curvature-aware Orthogonal | Very High               | Consistent                        | Scaled MoE Pretraining       |

## Advanced Iterations: MONA, Newton-Muon, and AdaMuon

The ecosystem of matrix-aware optimizers extends beyond standard Muon and SOAP, featuring variants designed to patch specific theoretical and empirical shortcomings.

**MONA (Muon with Normalized Acceleration):** While Muon is highly effective, it lacks element-wise adaptivity and can occasionally become trapped in sharp local minima, a trait common to first-order momentum approximations<sup>31</sup>. MONA addresses this by incorporating second-momentum estimation directly into the orthogonalized direction. By applying an acceleration term before momentum accumulation, MONA ensures the momentum buffer captures curvature-aware directions<sup>31</sup>. In MLX, the step time for MONA is virtually identical to Muon, as the added scalar operations (similar to AdamW's variance tracking) easily overlap with the heavy Newton-Schulz GEMMs. Empirically, MONA consistently outperforms both Muon and AdamW across multiple MoE benchmarks, spanning from 1B to 68B parameters<sup>31</sup>.

**Newton-Muon:** Theoretical critiques of standard Muon point out that it acts as an implicit Newton-type method that neglects the right preconditioning induced by the input data's second moment<sup>2</sup>. Newton-Muon introduces a triplet quadratic surrogate model that approximates the loss as a function of the gradient, output curvature, and the input data matrix<sup>2</sup>. By maintaining a running estimate of the input second moment and applying it during the step, Newton-Muon reaches target validation losses in 6% fewer iteration steps than standard Muon, with only a marginal 1.8% per-step wall-clock overhead, yielding an overall 4% reduction in total training time<sup>32</sup>.

**Muon-OGD and AdaMuon:** In continual learning scenarios where LLMs must adapt to new

tasks without catastrophic forgetting, Frobenius-norm projection methods often fail. Muon-OGD (Orthogonal Gradient Descent) bridges this gap by formulating parameter updates as a spectral-norm-constrained optimization problem with non-interference constraints, solved via dual iterations and Newton-Schulz matrix-sign approximations<sup>24</sup>. Similarly, AdaMuon seeks to integrate element-wise scaling directly into the orthogonal update, surpassing Adam's training efficiency by over 40% at scale without sacrificing Muon's stability<sup>33</sup>.

## Hybrid Regimes: Resolving Architectural and Stability Conflicts

A critical structural limitation of advanced matrix-based optimizers like Muon, SOAP, and MONA is their geometric rigidity; they rely on matrix decomposition math and are exclusively designed for 2-dimensional weight matrices (Linear layers, Attention projections, MLPs)<sup>13</sup>. A transformer model, however, contains numerous critical 1-dimensional parameters, including Layer Normalizations (RMSNorm), bias vectors, and massive token embeddings<sup>13</sup>.

Furthermore, empirical experiments reveal that attempting to apply pure Muon across an entire network (by artificially reshaping 1D tensors) often results in severe spectral-norm inflation. This inflation manifests as catastrophic gradient norm spikes, which destabilize the training run and invariably lead to the exploding attention logit crisis<sup>20</sup>.

### The Muon + AdamW Split Architecture

To circumvent these mathematical realities, state-of-the-art frameworks—including MLX-based trainers and frontier models like Qwen 3.8 Flash Next—uniformly deploy hybrid optimizer regimes<sup>20</sup>. In this hybrid regime, Muon or SOAP is explicitly assigned to the 2D linear map parameters, while a highly tuned AdamW (or a quantized variant like Gefen-X) governs the 1D parameters<sup>13</sup>.

This hybridization fundamentally dictates the step processing execution flow in MLX. During an `optimizer.update()` call, the MLX lazy computation graph diverges into two entirely distinct subgraphs<sup>8</sup>. The 1D parameters traverse an element-wise AdamW path, invoking fast, parallelized memory access patterns. Simultaneously, the 2D parameters traverse the compute-heavy Newton-Schulz path. Because MLX compiles these operations dynamically, the framework's Metal backend can theoretically interleave the memory-bound AdamW updates with the compute-bound Muon updates, maximizing the utilization of the M-series GPU<sup>5</sup>.

### Stability Solutions: MuonClip and Soft Musec

In environments where even the hybrid approach exhibits instability, researchers deploy constraint mechanisms. "MuonClip," utilized famously in the Kimi K2 1T-parameter model, applies QK-norm clipping to the Muon updates to stabilize massive pretraining runs<sup>34</sup>. However, in smaller-scale fine-tuning environments (such as a 4B parameter model fine-tune), MuonClip was shown to underperform the standard Muon+AdamW hybrid, resulting in lower token accuracy and slower convergence due to the overly aggressive clipping<sup>34</sup>.

Alternatively, Soft Musec offers an architecture-agnostic mechanism for stabilizing Muon training. It utilizes a smooth spectral saturation function approximated by coupled Newton-Schulz iterations, acting as an efficient soft spectral clipping mechanism that avoids

expensive SVDs while preventing the spectral norm inflation that plagues raw Muon<sup>20</sup>.

## Empirical Dynamics in Apple Silicon Deployments

The theoretical differences in step times between AdamW and Muon manifest distinctly in practical MLX workloads, ranging from autonomous research loops to specialized fine-tuning applications.

### Karpathy's Autoresearch Loop on MLX

In automated AI research paradigms—such as Karpathy's autoresearch loop ported to Apple MLX—an AI agent iteratively tweaks model code, trains for a strictly fixed wall-clock time budget (e.g., exactly 5 minutes or 300 seconds), and evaluates the validation bits-per-byte (BPB) score<sup>41</sup>. In these strictly time-bound environments, the optimizer's per-step wall-clock execution speed is pitted directly against its per-step convergence efficiency. If a faster step execution (AdamW) allows the model to process 30% more tokens in 5 minutes, it theoretically might outperform a slower, geometrically superior optimizer (Muon) that processes fewer tokens. However, empirical measurements on Apple Silicon dictate otherwise. When scaling LLMs, the computational overhead of the Muon step grows, but its convergence advantage scales disproportionately faster. An untrained 11.5M parameter model running on an Apple M1 Pro (16GB UMA) achieved a validation score of 2.371 BPB after just 55 optimizer steps in a 5-minute window<sup>41</sup>. In NanoGPT speedrunning benchmarks, Muon improved training speed by 35% over AdamW, reaching GPT-2 XL level performance approximately 25% faster in total wall-clock time, definitively proving that the reduction in required tokens heavily outweighs the increased step latency<sup>13</sup>. The integration of Gram Newton-Schulz further stretches this advantage, cutting the Muon overhead to virtually negligible levels<sup>32</sup>.

### Fine-Tuning and Quantization Aware Training (QAT)

For Mac-based fine-tuning utilities like mlx-lm-lora, the choice of optimizer interacts deeply with Quantization Aware Training (QAT)<sup>43</sup>. QAT projects trainable weights onto a quantized grid after each optimizer update, simulating quantization effects during training to improve post-training model robustness<sup>43</sup>. In Low-Rank Adaptation (LoRA) based fine-tuning, the parameter matrices are split into low-rank adapters. Because these adapters have highly rectangular aspect ratios (e.g., a rank of 16 against a hidden dimension of 4096), applying standard Newton-Schulz is mathematically highly inefficient<sup>23</sup>.

By leveraging the Gram Newton-Schulz variant in these MLX repositories, the orthogonalization

computation occurs entirely within the tiny  $16 \times 16$  subspace<sup>23</sup>. This renders the Muon step virtually instantaneous. For environments mirroring the constraints of the Apple-MLX-llm-gpu-8 target—where memory is tight and compute must be maximized—combining QAT, 8-bit Gefen-X for 1D vectors, and Gram Muon for 2D LoRA matrices represents the absolute pinnacle of current LLM optimization efficiency on Apple Silicon<sup>7</sup>.

| Benchmark Task | Model Scale | Best AdamW Score | Best Muon Score | Best Hybrid/MONA Score |
|----------------|-------------|------------------|-----------------|------------------------|
|                |             |                  |                 |                        |

|                         |             |               |                 |                                         |
|-------------------------|-------------|---------------|-----------------|-----------------------------------------|
| <b>MMLU-FewShot</b>     | 1B - 3B MoE | 0.6218        | 0.6281          | <b>0.6373</b><br>(MONA)                 |
| <b>GSM8K (CoT)</b>      | 1B - 3B MoE | 0.4426        | 0.4550          | <b>0.4734</b><br>(MONA)                 |
| <b>MBPP+</b>            | 1B - 3B MoE | 0.534         | 0.548           | <b>0.7857</b> (MONA pretrain)           |
| <b>NanoGPT Speedrun</b> | 1.5B        | Baseline time | -25% wall-clock | <b>-29% wall-clock</b><br>(Newton-Muon) |

## Conclusion

The extensive analysis of optimizer step processing times within the Apple MLX framework reveals a definitive and irreversible transition in the field of LLM training. The industry is moving from memory-bound, element-wise optimization paradigms to compute-bound, matrix-geometric descent algorithms.

AdamW, and its highly compressed 8-bit derivatives such as Gefen-X and AdamWGPU, remain the most universally stable, generalized, and lowest-latency choices on a per-step basis. They perfectly exploit the high memory bandwidth and zero-copy advantages inherent in Apple's Unified Memory Architecture, executing swiftly via custom Metal kernels. However, their scalar nature forces models to consume vastly more tokens to reach convergence.

Conversely, Muon and GEMM-based preconditioned optimizers like SOAP offer a profound paradigm shift. They trade increased step processing time and algorithmic complexity for vastly superior sample efficiency. While standard Muon introduces severe cubic-scaling bottlenecks due to its reliance on rectangular matrix Newton-Schulz iterations, algorithmic breakthroughs like the Gram Newton-Schulz technique successfully eliminate this overhead. By mathematically isolating the iterations to small, square Gram matrices, GNS synergizes perfectly with the symmetric Metal GEMM capabilities of Apple Silicon, cutting the FLOP requirement by up to 50%.

Ultimately, for modern LLM workflows on Apple Silicon—ranging from 8-bit quantized local fine-tuning environments like Apple-MLX-llm-gpu-8 to autonomous architectural search loops—the optimal configuration is unequivocally the hybrid regime. Deploying Gram Newton-Schulz-accelerated Muon for the optimization of 2D parameter projections, running in parallel with a quantized, decoupled AdamW for 1D parameter updates, provides the highest yield of validation loss reduction per unit of wall-clock time. This hybridization successfully navigates the complex interplay between the MLX lazy computation graph, the strict memory constraints of local workstations, and the silicon realities of the M-series GPU.

## Works cited

1. SOAP, Muon, and Beyond: Pushing LLM Pretraining Scales - arXiv, <https://arxiv.org/html/2607.20548v1>
2. The Newton–Muon Optimizer - arXiv, <https://arxiv.org/html/2604.01472v1>
3. Soap: Improving and stabilizing shampoo using adam. - arXiv, <https://arxiv.org/pdf/2409.11321>
4. Goekdeniz-Guelmez/MLX-Benchmark-V2 · Datasets at Hugging Face, <https://huggingface.co/datasets/Goekdeniz-Guelmez/MLX-Benchmark-V2>
5. Ruby Finally Gets a Real Machine Learning Framework - Skryl, <https://skryl.org/posts/ruby-deserves-ml>
6. Random Request Failures · Issue #276485 · microsoft/vscode - GitHub, <https://github.com/microsoft/vscode/issues/276485>
7. Gefen-X: A Low Memory Optimized Stochastic Optimizer (Fork, <https://github.com/thad0ctor/Gefen-X>
8. Optimizers — MLX 0.32.2 documentation, <https://ml-explore.github.io/mlx/build/html/python/optimizers.html>
9. Compilation — MLX 0.32.2 documentation, <https://ml-explore.github.io/mlx/build/html/usage/compile.html>
10. mlx.core.fast.metal\_kernel — MLX 0.32.2 documentation, [https://ml-explore.github.io/mlx/build/html/python/\\_autosummary/mlx.core.fast.metal\\_kernel.html](https://ml-explore.github.io/mlx/build/html/python/_autosummary/mlx.core.fast.metal_kernel.html)
11. Custom Metal Kernels — MLX 0.32.2 documentation, [https://ml-explore.github.io/mlx/build/html/dev/custom\\_metal\\_kernels.html](https://ml-explore.github.io/mlx/build/html/dev/custom_metal_kernels.html)
12. Muon Optimizer Accelerates Grokking - arXiv, <https://arxiv.org/html/2504.16041v1>
13. Using Muon Optimizer with DeepSpeed - PyTorch, <https://pytorch.org/blog/using-muon-optimizer-with-deepspeed/>
14. A Fast, Hardware-Aware Newton-Schulz Algorithm for Muon | Tri Dao, <https://tridao.me/blog/2026/gram-newton-schulz/>
15. H/T Ben Dickson FlashOptim: Optimizers for Memory Efficient, <https://www.facebook.com/groups/DeepLearnng/posts/4487707171462906/>
16. Optimizer: Adam, AdamW, SGD — the gradient-update wrappers, <https://zeroentropy.dev/concepts/optimizer/>
17. Daily Papers - Hugging Face, <https://huggingface.co/papers?q=Muong%20optimizer>
18. Awesome-Optimizers/README.md at main - GitHub, <https://github.com/JiwenJ/Awesome-Optimizers/blob/main/README.md>
19. Benchmarking Optimizers for Large Language Model Pretraining, <https://wandb.ai/semenov-andrei-v/llm-optimizer-benchmark/reports/Benchmarking-Optimizers-for-Large-Language-Model-Pretraining--VmlldzoxNDc1NDM0Ng>
20. Musec: MomentUm SpEctral Clipping for Stable Muon-type Training, <https://arxiv.org/html/2609.11655v1>
21. Inside nanochat Part 3: Understanding Optimizers - Medium, [https://medium.com/@akdemir\\_bahadir/inside-nanochat-part-3-understanding-optimizers-7f989d407153](https://medium.com/@akdemir_bahadir/inside-nanochat-part-3-understanding-optimizers-7f989d407153)

22. Muon is Scalable for LLM Training - arXiv, <https://arxiv.org/pdf/2502.16982>
23. Dion3: Full-Stack Orthogonal Updates - alphaXiv, <https://www.alphaxiv.org/abs/2608.11612>
24. Muon-based Spectral Orthogonal Gradient Projection for LLM ... - arXiv, <https://arxiv.org/pdf/2605.08949>
25. Spectral Scaling Laws of Muon - arXiv, <https://arxiv.org/html/2606.04058v1>
26. Muon is Scalable for LLM Training - arXiv, <https://arxiv.org/html/2502.16982v1>
27. MLX GPT — Training GPT-2 on Two Mac Minis with Apple MLX, <https://mlxgpt.com/>
28. SOAP: Improving and Stabilizing Shampoo using Adam - arXiv, <https://arxiv.org/html/2409.11321v1>
29. Towards Efficient Optimizer Design for LLM via Structured ... - arXiv, <https://arxiv.org/pdf/2502.07752?>
30. Understanding and Improving Shampoo and SOAP via Kullback, <https://arxiv.org/pdf/2509.03378>
31. MONA: Muon Optimizer with Nesterov Acceleration for Scalable, <https://arxiv.org/html/2605.26842v2>
32. The Newton-Muon Optimizer - alphaXiv, <https://www.alphaxiv.org/abs/2604.01472>
33. Muon Variant for Efficient Finetuning of Large Language Models, <https://arxiv.org/pdf/2511.06086>
34. Muon vs MuonClip vs Muon+AdamW for Fine-Tuning - Hugging Face, <https://huggingface.co/blog/KingNish/optimizer-part1>
35. The Muon optimizer has demonstrated remarkable performance in, <https://www.facebook.com/groups/big.data.egypt/posts/4235713443354491/>
36. Qwen3.8-Flash-Next: A Deep-Dive into Alibaba Qwen4 Architecture, <https://local-ai-zone.github.io/blog/qwen3-8-flash-next-deep-dive.html>
37. What Is Qwen 3.8 Flash Next? 125B MoE - Kie.ai, <https://kie.ai/blog/what-is-qwen-3-8-flash-next>
38. Kimi's K2 Opensource Language Model Supports Dynamic ... - InfoQ, <https://www.infoq.com/news/2025/11/kimi-k2-open-source-moe-ai/>
39. Muon vs MuonClip vs Muon+Adamw : r/LocalLLaMA - Reddit, [https://www.reddit.com/r/LocalLLaMA/comments/1pj30m1/muon\\_vs\\_muonclip\\_vs\\_muonadamw/](https://www.reddit.com/r/LocalLLaMA/comments/1pj30m1/muon_vs_muonclip_vs_muonadamw/)
40. [2603.14315] Enhancing LLM Training via Spectral Clipping - arXiv, <https://arxiv.org/abs/2603.14315>
41. Your MacBook Can Do Autonomous AI Research Now - Naman Goyal, <https://namangoyal.com/blog/2026/autoresearch-mlx/>
42. Run Karpathy's autoresearch on a GPU VM: 100 Experiments, <https://www.spheron.network/blog/karpathy-autoresearch-spheron-gpu/>
43. Goekdeniz-Guelmez/mlx-lm-lora: Train Large Language Models on, <https://github.com/Goekdeniz-Guelmez/mlx-lm-lora>
44. mlx-lm-lora - PyPI, <https://pypi.org/project/mlx-lm-lora/>